Painter: Strategies for the Open Feature Questions

A product and architecture research report

Research synthesis; recommendations are not accepted product decisions.

Scope. This report examines the choices implied by the project brief's node-graph brushes, graph-based composition, vector-first content, undo/history, errors, editing focus, performance, persistence, and remaining MVP interaction questions. It compares publicly documented behavior in Krita, Clip Studio Paint, Procreate, Photoshop, Blender, TouchDesigner, and the SVG standard. Product manuals establish documented user-facing behavior; they do not establish competitors' internal implementation architecture.

1. Executive summary

Painter already has several sound foundations: document-space stroke data, deterministic derived pencil splats, a versioned save format, and a multi-resolution cache that builds tiles lazily. The current document is nevertheless a concrete, ordered list of immutable strokes. The tile cache depends on that list being append-only and on the current pencil effect only adding alpha. That is a good prototype contract, not a general node graph, arbitrary vector scene, or undo system.

The safest near-term course is to keep shipping the current macOS workflow and avoid building speculative general frameworks. Before implementing the killer features, settle the *semantics* below. Then make the smallest system that demonstrates one real brush graph and one real composition graph, and measure it before adopting compilation or complex cache machinery.

Area	Recommended direction	Reason / do not do yet
Brushes	Define a typed, acyclic graph contract; first evaluate a small graph as an immutable execution plan at stroke-sample time. Store brush identity/version and the resolved configuration used by each stroke.	Supports the stated "all brushes are graphs" goal while postponing a JIT/compiler until profiling proves it necessary. Do not store only a mutable preset name.
Composition	Keep brush graphs distinct from document-composition graphs. Prototype a directed acyclic graph (DAG) with explicit sources and one active output; previewing and editing are separate state.	A layer stack is simpler and familiar; a general node editor is costly. A constrained graph has a testable end-to-end path without claiming arbitrary graph support.
Vector / raster	Define source content, evaluated content, and disposable render caches separately. Treat raster images/textures as raster sources when introduced, not as "vectors" by vocabulary.	"Vector" can mean editable geometry, not that every algorithm or asset is resolution-independent. Rendering caches already produce raster tiles.
Undo	Choose visible undo semantics first. A command/history cursor is the lowest-risk first implementation; use an append-only event log only if durable replay/history is a requirement.	An append-only log does not by itself define undo, redo, erasing, compaction, or cache invalidation.
Errors	Keep per-file-operation errors recoverable; add central logging for framework and uncaught asynchronous errors. Define fatal classification and durable-log guarantees before exit behavior.	The brief says what classes should do, but not which failures are fatal or how to prove logs reached durable storage.

The report's highest-value immediate work is therefore specification, not a graph editor: clarify the brief where it conflicts with established behavior, write observable acceptance scenarios, and create a roadmap task for the uncovered global error-reporting requirement after the current T-005 task. This report itself is not an ADR and does not change the project's accepted choices.

2. Existing implementation and constraints

The current architecture stores `CanvasDocument` as an ordered list of immutable `CanvasStrokes`. Each stroke contains document-space points and brush weights. `CanvasDocument.toJson/fromJson` persist a version-1 JSON document with that stroke list. `CanvasController` owns document mutation and transient viewport/input state. The implementation's renderer derives gaussian pencil splats and rasterizes document-space tiles lazily.

Two existing implementation assumptions are especially relevant:

- `TileCache` tracks how many leading strokes have been applied to each tile. It incrementally draws new strokes onto old tile images; its overview explicitly depends on append-only strokes and additive pencil ink. Replacing or changing earlier content therefore needs a different invalidation/rebuild strategy.
- The app catches file-operation errors, logs their error and stack trace using `debugPrint`, displays a details-expandable dialog, and does not replace the current document on a failed open. The app does not currently install application-wide handlers for non-file failures; this gap is noted in the source and project status.

The saved format's integer version gives the project a migration boundary, but version 1 cannot preserve node graphs, graph revisions, stable editable object identifiers, or an explicit brush configuration. Do not infer these from current fields.

3. Brush graphs

3.1. What "all brushes are node graphs" must mean

A brush graph describes the transformation from sampled input and brush configuration to mark geometry or other declared brush output. It is not the same graph as the graph that composes a document. Keeping these two graph families separate avoids an ambiguous graph whose nodes sometimes run once per sample and sometimes render whole document branches.

A useful first contract defines:

- *Inputs*: position, pressure, tilt, azimuth, speed, elapsed stroke distance, random seed, and user parameters. Each input needs units, valid ranges, and a rule for missing platform data.
- *Outputs*: at minimum the mark geometry or mark parameters, opacity/coverage, and any color/material data the product commits to support. State whether nodes output points, curves, splats, or a generic primitive set.
- *Types and units*: scalar, vector, color, curve/profile, geometry, and texture types only when they have concrete semantics. Specify conversions rather than silently converting socket values.
- *Evaluation*: what happens at each pointer event versus each resampled stroke sample; what state is held between samples; whether stochastic nodes are deterministic from a stored seed.
- *Graph validity*: whether the graph must be acyclic, how missing inputs and type mismatches are handled, how disabled/unknown node types load, and what happens when evaluation fails.
- *Versioning*: stable node type identifiers and graph schema/version; define whether a stored stroke references a live brush, embeds a brush snapshot, or contains resolved output parameters.

A graph should not be promised as "all brushes" until it can represent the product's intended hard brush, pencil, and at least one materially different brush without special-case execution paths. The point is behavioral uniformity, not a node canvas for its own sake.

3.2. Evaluation and compilation options

Approach	Strengths	Costs / decision trigger
Fixed engines with presets	Simple execution and tuning; each brush family can have purpose-built behavior. Krita documents multiple brush engines and presets, with engine-specific options and pressure/sensor curves. Procreate exposes a large parameter editor with live drawing-pad preview and per-setting pressure curves.	This is common and comprehensible, but it conflicts with a literal "all brushes are node graphs" product goal. Do not call presets graphs unless graph editing and evaluation are genuinely the same system.
Interpret a graph per sample	Directly reflects authored graph semantics; easy to inspect and alter. It is the simplest real route to validate graph design.	Cost scales with graph size × number of stroke samples. Keep the evaluator allocation-conscious; profile actual large graphs and long strokes rather than guessing.
Compile graph to an execution plan	Can validate types/cycles once, order operations topologically, and remove editor-only structure from runtime evaluation.	Adds compiler/debugging complexity. Compile to ordinary operations first; do not jump to native code/JIT without evidence that the plan misses a measured budget.
Hybrid: graph authoring + specialized kernels	A graph describes composition and parameter wiring; expensive primitive generation uses tested, specialized kernels.	A good probable endpoint, but premature as a generalized plugin ABI. Introduce only when the first graphs demonstrate a repeated bottleneck or complex kernel.

The project brief already says graph technology is TBD and to "go with naive route for now." A conservative interpretation is an interpreter or validated in-memory execution plan, with performance measurements before compilation work. "Naive" does not justify a per-frame rebuild or allocation-heavy work: sampling, deterministic random seeds, and reusable buffers still have ordinary performance requirements.

3.3. Brush identity and editing old strokes

Three persistence strategies have materially different behavior:

- *Live reference*: each stroke stores a brush asset ID; editing the brush changes every stroke that references it. This gives linked, global editability, but makes an old document sensitive to a missing or changed brush asset.
- *Embedded snapshot*: each stroke stores the exact graph and parameters used. This keeps documents portable and old marks stable, but copies data and makes intentional global brush updates a separate operation.
- *Reference plus frozen revision*: a project-owned brush asset has a stable ID and immutable revision; strokes record ID + revision, with a fallback embedded snapshot or a clear missing-asset error. This supports reuse and reproducibility, but requires asset lifecycle and migration policy.

For an MVP, the embedded snapshot or immutable revision is safer than a mutable name/reference. The brief should decide whether "editing past strokes" means changing the input path, the brush parameters used for that stroke, a shared brush definition, or all three. State whether graph updates affect old marks by default, only after an explicit "re-evaluate" action, or never.

Procreate and Krita are useful contrasts, not proof that Painter should copy their internals: their official manuals describe brush settings/presets and editable sensor curves rather than exposing every brush as a user-facing general-purpose node network. Their live test/preview surfaces suggest an important acceptance requirement for Painter: graph settings need a fast, representative preview and a way to compare pressure response before applying a brush to artwork.

4. Composition graphs and focused previews

4.1. Three viable composition models

Model	How it works	Trade-off
Ordered layer/tree model	Canvas content is grouped and composited in a predictable back-to-front order. Nodes, if present, are mostly modifiers attached to a layer or group.	Familiar to painting users; simple ordering and visibility. Less expressive for branching, reuse, and multiple intermediate outputs.
General directed graph	Typed nodes connect sources, transforms/modifiers, and outputs; a node can feed multiple downstream branches.	Expressive and aligns with the TouchDesigner/Blender inspiration. Requires graph validation, dependency tracking, preview rules, and more discoverable authoring.
Hybrid graph with ordered containers	A graph wires canvas/group sources and effects; each drawable container retains an ordered sequence of strokes or objects.	Preserves natural drawing order and efficient local editing while allowing branching at meaningful points. More concepts and UI than a pure stack.

Krita documents a conventional layer stack composited bottom-to-top, with groups, masks, filters, blending modes, and separate vector layers. Photoshop's public documentation likewise describes non-destructive techniques such as adjustment layers, masks, and Smart Objects. Clip Studio Paint puts editable vector strokes in vector layers rather than requiring an arbitrary network for basic ink. These are strong examples of the usability of ordered containers and specialized non-destructive operations.

Blender demonstrates a more constrained graph pattern: Geometry Node Groups are used in a modifier/tool context; its compositor has an explicit Group Output that determines which values leave a group. Grease Pencil modifiers can be stacked non-destructively and Geometry Nodes can provide custom modifiers. TouchDesigner documents a pull-based computation graph that cooks only when a node is requested and has a reason to update. These examples

show that "graph" is not one design: type/context, output contract, evaluation trigger, and user intent need to be explicit.

4.2. Recommended first graph contract

Treat the composition as a DAG initially, with explicit data types and one designated document output. Permit multiple consumers of a source. Reject cycles with a readable validation error rather than guessing at feedback semantics. A constrained initial vocabulary might include canvas/source, ordered drawing content, transform, modifier, composite, and output nodes; do not promise these node types until each has a defined input/output contract.

Keep three concepts distinct:

1. *Document output*: the graph result shown in the normal Canvas view.
2. *Focused preview*: the output of a selected node, shown in a preview surface; this is inspection, not necessarily a change to the document output.
3. *Edit target*: the canvas/source that receives new strokes. Selection/focus alone should not silently redirect input.

Specify what "the last canvas node selected is visible in the Canvas window for editing" means when that node is downstream of transforms or modifiers. If editing happens in source coordinates, show the transform relationship and keep viewport/document coordinates consistent. If editing happens on an evaluated result, define whether the action edits source data, adds a branch, or applies/bakes a modifier.

SVG provides a useful standards-level analogy: its document tree and rendering tree are not one-to-one, reused graphics may render in multiple places, and painter order determines overlap. It is not a blueprint for a painting application's graph, but it reinforces the value of separating canonical document structure from the evaluated/rendered representation.

4.3. Preview, evaluation, and invalidation

A focused preview can become a performance trap if every interaction reevaluates the whole graph. Decide which edits invalidate which outputs, whether previews are synchronous or asynchronously refreshed, how stale results are labeled, and whether a selected node preview can temporarily lag behind. A useful contract is: graph edits mark that node and dependent descendants dirty; unrelated branches remain reusable; a result is displayed only with the graph revision that produced it.

TouchDesigner's documented "cook only when requested and when there is a reason to cook" provides a relevant lazy-evaluation model. Painter should not copy TouchDesigner's full runtime; the applicable lesson is to separate invalidation from evaluation and to compute only demanded graph outputs. A node's user-visible preview is one such demand; the final Canvas is another.

5. Vector-first content, raster sources, and caches

"Everything is a vector" currently overstates what an artist-facing application can guarantee. The current pencil is generated from point samples and rendered as splats, then stored in raster tile images. A future textured brush may use raster stamps; imported images are pixels; some filters are naturally defined in pixel space. Those can still coexist with vector-editable source data if the boundary is explicit.

Define three layers of representation:

- *Canonical content*: editable stroke inputs, geometry, graph definitions, parameters, and explicit raster assets. This is what the saved document must preserve.
- *Evaluated content*: the result of applying brush graphs, geometry transforms, modifiers, and compositing. It may still be geometry, or may include raster surfaces where the operation requires them.
- *Derived render cache*: tile images, previews, generated splats, thumbnails, and intermediate textures. Disposable and rebuildable; not the source of truth.

Potential source models include sampled input paths, fitted paths/curves, generated mark geometry, or a hybrid. Sampled input paths preserve observed stylus data and are simplest; path fitting reduces storage and makes control-point editing easier but changes captured input; storing generated geometry freezes appearance but can weaken future brush editing. The brief should say which representation remains authoritative and which may be regenerated.

The W3C SVG rendering model separates a document model from its rendering tree and defines document-order painting; the W3C compositing specification defines image combination operations. Those standards are useful

references for tree/render and compositing vocabulary, but neither decides Painter's file model or guarantees all effects can remain vector. The product brief should explicitly allow or forbid: embedded raster images, raster brush stamps, raster-only filters, finite-resolution exports, and destructive "apply/bake" operations.

Recommended invariant: caches never appear in the canonical document format as required content. If a cache is serialized later for startup speed, it must be optional, versioned, checksummed or safely invalidatable, and regenerable from canonical data. "Arbitrary resolution" should be qualified by which operations preserve it and which raster sources or filters impose resolution limits.

6. Undo, redo, erasing, and append-only history

6.1. First define the user-visible behavior

The phrase "undo everything" can refer to at least three different contracts: undo only recent actions within the open session; navigate a retained project history after saving/reopening; or reconstruct every past document state indefinitely. These have very different memory, persistence, and performance costs. The brief should select one.

Also specify the unit of an undo step. One pointer stroke is a natural default for painting. Brush/graph edits, node connection changes, transforms, selections, open/save operations, and viewport gestures may have different policies. Define redo truncation after new edits, grouping/coalescing of repeated edits, undo limits, and whether saves preserve the undo stack.

6.2. Eraser is a compositing decision

Appending an eraser-shaped mark does not automatically erase previously rendered content. The system must define how it combines with prior content: destination-out/subtractive compositing, a mask applied to a source or group, a vector erase operation that splits/changes geometry, or a special ordered operation interpreted during evaluation. Each choice changes what it means to move, disable, undo, or re-evaluate earlier marks.

Clip Studio Paint's documentation demonstrates that "vector eraser" itself can mean distinct behaviors: erase touched parts, erase to intersections, or erase an entire line. These are product operations with different geometric consequences, not implementation details to postpone indefinitely. Painter's brief needs to state which erase modes exist in the first supported case and whether erasure targets a stroke, a canvas, or a graph branch.

6.3. History implementation choices

Approach	Semantics	Trade-off
Command stack + snapshots/deltas	Record reversible user actions; undo moves a history cursor backward; a new action after undo drops the redo branch.	Conventional and easy to explain/test. Memory and custom reversible logic are concerns; history is usually a session feature unless explicitly persisted.
Append-only event log	Record immutable operations; current state is a projection at a chosen history position. Undo can append a compensating event or move a persisted active-history pointer.	Supports replay, audit, durable history, and perhaps collaboration. Requires stable IDs, deterministic replay, explicit branching/compaction, versioned events, and a rebuildable projection. "Append-only" alone supplies none of these.
Immutable document snapshots	Each action produces a new document state; history retains selected prior states, potentially with structural sharing.	Simple state transitions and useful for editing graphs; full snapshots can be expensive without sharing/deltas and do not automatically make persistence append-only.
Hybrid	In-session command history, durable canonical document state, optional checkpoints/event log later.	Often safest for an early single-user editor; do not promise durable undo until persistence and recovery are designed.

Krita's user manual describes a visible Undo History docker and "cumulative undo" that merges similar stroke states under configurable timing/count rules. Blender documents an undo timeline that can jump to earlier actions, truncates the redo history when a new action is made, and loses its action history when Blender quits—even if the

file was saved. These are documented user-visible behaviors, not claims about either application's internal storage design. They show that powerful undo need not mean an indefinitely durable append-only project database.

Recommendation: implement conventional reversible user-facing undo first, unless durable replay across sessions is a product requirement. If append-only persistence is non-negotiable, describe it as a versioned operation log with explicit branch and compaction semantics; treat undo as a first-class, persisted event or active-history pointer. Do not make the current `CanvasDocument` list pretend to be that log.

6.4. Consequences for the tile cache

The present cache's incremental append path is only valid when earlier source content never changes and new content is purely additive. Undoing the latest append could be supported by discarding/rebuilding affected tiles; undoing an old stroke, changing graph parameters, reordering branches, applying a mask, or changing a compositing operator may affect a large region or the whole output. The brief should define expected document sizes and responsiveness budgets before choosing full rebuild, region invalidation, dependency-aware partial recompute, or persistent tile snapshots.

A likely evolution is a cache keyed by source/graph revision and output region, with dependency tracking from edited source nodes to affected output tiles. This is a future design direction, not a request to generalize `TileCache` before graph and edit semantics exist. The current implementation's document-identity revision invalidation is safe for replacement but does not by itself represent fine-grained graph dependencies.

7. Persistence, compatibility, and editability

The current `.paint` format is UTF-8 JSON with format identifier `painter-document` and version `1`; it serializes point-based strokes and weights. It is a small, inspectable prototype format. Treat the next change in canonical structure as a schema decision, not an incidental serialization edit.

Before adding nodes/brushes, specify:

- Stable IDs for nodes, connections, canvas sources, brush definitions, and editable strokes; specify ID lifetime on duplicate/copy/paste.
- Whether ordering is explicit in the graph, in a drawing container, or both.
- How unknown node types and newer document versions behave: reject with a clear error, open read-only, substitute a visible unsupported node, or provide a fallback render.
- Whether brush graphs are embedded, referenced, or pinned to immutable asset revisions; how missing assets are handled.
- What counts as canonical source versus optional generated output; whether save waits for in-flight evaluations or stores only canonical data.
- Migration policy for v1 files and the exact round-trip invariant: same input geometry, same appearance, or same editable structure.

A conservative compatibility goal is: every file version either opens with a defined faithful fallback or fails before replacing the current document; saving must not silently discard unsupported data. Tests should cover migration, stable IDs, missing graph assets, deterministic seeded output, and failure preservation—not only JSON shape.

8. Errors and application failure policy

The brief already specifies three behaviors: file-operation failure produces a details-expandable popup and logs without terminating; non-fatal internal errors/exceptions log without terminating; fatal failures terminate only after relevant information is successfully logged. The first behavior exists for open/save; the latter two are not implemented centrally.

The missing specification is the classification and safety boundary:

Class	Candidate behavior	Questions to settle
Cancellation / expected result	No error dialog; no error-level log.	Which failures are ordinary cancellation or unavailable optional input?
Recoverable user operation failure	Log operation, error, stack trace, and safe context; show actionable error; preserve the last valid document and keep the app usable.	Which errors can be retried? Which user-facing text should avoid exposing paths or system detail?
Internal recoverable failure	Log with context and stack; keep running only if the affected subsystem can be left in a safe state. Report a visible failure where silent stale/incorrect content would mislead.	What state can be rolled back or marked unavailable? Is a per-node error badge better than a modal?
Fatal / state integrity lost	Synchronously write and flush the relevant report, then exit or enter a defined recovery screen.	Which conditions are fatal? What does "successfully logged" mean: OS log accepted, file flushed, or crash-report service acknowledged?

Flutter's official guidance distinguishes framework-caught errors (`FlutterError.onError`) from errors outside framework callbacks (`PlatformDispatcher.instance.onError`) and shows how to customize both in `main()`. It also notes the default framework behavior reports errors through Flutter's presentation path, while non-callback errors use the platform dispatcher. This supports a small centralized reporting seam as the first implementation step. It does *not* resolve fatal classification, guarantee durable logging, or make it safe to resume after arbitrary failures. Retain the current local file-operation catches because they have useful operation context and preserve the document transaction boundary; route their reporting through the same logger once introduced.

The implementation should avoid logging document contents, brush graph payloads, or other sensitive data. Define whether file paths are logged and whether the app writes a local diagnostic file in addition to terminal/platform logs. Do not swallow an error merely to satisfy "keep running"; return a handled status only when the app has actually taken responsibility for reporting and recovery.

9. Artist-facing defaults and workflow questions

9.1. Brush size and pressure

Current prototype values—`8` logical-pixel brush diameter and linear pressure mapping to weight `0.05–1.0`—are documented as provisional. Krita's brush settings allow sensor curves and different calculation modes; Procreate documents per-setting pressure curves and a live drawing pad. Both make clear that the input-to-mark mapping is a user-facing brush characteristic, not just a platform constant.

The brief should distinguish brush diameter in document units from screen pixels; state pressure normalization and behavior at zero/missing pressure; define whether pressure affects size only for every brush or is merely the MVP pencil rule; state how tilt, speed, barrel rotation, and stylus identity are ignored or used; and include a physical-device validation scenario. Prefer a testable default plus an adjustable curve if artists need calibration; do not select a curve based solely on synthetic pointer tests.

9.2. Pan, zoom, and rotation

The current brief says mouse input can pan, zoom, and rotate, while the architecture has chosen mouse buttons/wheel and trackpad two-finger gestures. Make the product contract spell out the control, modifier key, cursor/focal anchoring, scroll direction, rotation direction, and behavior when input devices differ. Keep documented controls stable unless a usability test demonstrates a better assignment; test trackpad and mouse separately on macOS hardware.

9.3. Open over unsaved work

The product brief lists this as unresolved, but ADR 0002 already records the current choice: opening replaces the document without an unsaved-changes prompt. The brief must either accept that behavior explicitly for the MVP or change it. Options are immediate discard, confirmation prompt, or recoverable autosave. The safe default for

artist work is a prompt once dirty tracking exists, but it is not free: the app needs a reliable dirty-state invariant, dialog choices, and tests for save failure/cancel. Until then, explicitly document that Open discards unsaved work so behavior is not mistaken for an omitted feature.

9.4. Focused node preview and Canvas editing

Define selected node versus active edit target, whether previews are live or on-demand, how the app indicates a stale preview or evaluation failure, and what happens to the viewport when switching targets. Acceptance criteria should assert that selecting an output node does not silently edit its source and that choosing an editable canvas node has an observable, reversible result.

10. Decisions to make before the next feature implementation

Order	Decision	Why / concrete deliverable
1	Reconcile the project brief with implemented Open/Save behavior and decide whether Open discards unsaved changes in the MVP.	The current brief still says first-save behavior is unspecified although the ADR and code establish it; Open's discard behavior is also already implemented.
2	Define the first brush graph's input/output and versioning contract; define how a stroke binds to a brush revision.	Enables a single representative graph without premature general-purpose runtime work.
3	Define graph composition's node types, DAG/cycle policy, output selection, edit target, and preview behavior.	Turns "like TouchDesigner and Blender" into behavior that can be tested; clarifies whether canvas nodes are sources, containers, or both.
4	Choose undo scope and eraser compositing semantics; state whether history persists across save/reopen.	Makes append-only, cache invalidation, and document storage decisions coherent.
5	Define canonical vector/raster categories and the guarantee meant by "arbitrary resolution."	Avoids promising scalable output for raster stamps, images, or pixel-space filters.
6	Define error classes, safe recovery states, and the evidence required before fatal exit.	Makes the existing errors section implementable and testable without choosing an arbitrary logging library.
7	Set benchmark targets and representative documents/graphs before optimizing or compiling.	Separates observed performance needs from imagined ones; cache complexity needs a workload and latency target.

Once decisions 2–4 are made, create separate roadmap tasks for brush-graph evaluation and graph composition, each with a narrow vertical slice and acceptance tests. Keep the error-handler task independent. Do not combine all the killer features into one architecture milestone.

11. Suggested observable acceptance scenarios

The brief should make these kinds of scenarios concrete before code is committed:

- **Brush graph:** a pressure-driven graph produces a measurably different stroke at low and high pressure; its output is deterministic for a fixed seed; saving/reopening gives the specified same/different appearance; an invalid graph has a visible, non-destructive failure.
- **Graph persistence:** unknown graph schema/node behavior is specified; a failed open leaves the current document unchanged; older supported files retain their marks under documented migration rules.
- **Composition:** a source -> modifier -> output graph produces the expected image; changing a source updates only dependent outputs; a node preview displays that node's output while the final Canvas output remains defined; cycles are rejected with a useful explanation.
- **Vector editing:** one old stroke can be edited or re-evaluated according to the chosen brush-binding policy; any raster-derived cache is recreated from canonical sources; zooming does not falsely promise detail beyond raster inputs.

- **Undo/redo:** drawing, erasing, editing a graph connection, undo, redo, saving, and reopening behave according to the chosen history persistence policy; a new edit after undo has an explicit redo-branch result.
- **Erasing:** each supported erase mode has a defined target and observable effect; undo restores exactly the affected content; tile output has no stale marks or gaps after the operation.
- **Errors:** a failed file read/write and a recoverable internal error are each logged with a stack trace, leave the last valid document available, and do not terminate; fatal behavior is tested only after durable logging and classification are specified.
- **Performance:** representative graph size, sample count, document size, frame/preview latency, and memory budget are recorded; benchmarks exercise release-relevant paths, not only UI-thread display-list recording.

These are scenario templates, not claims that the project has already met them. The existing project brief requires user-visible macOS demonstration as well as automated coverage; synthetic tests cannot substitute for physical stylus and trackpad evidence.

12. Comparative field notes

Software / standard	Documented approach relevant here	Useful lesson for Painter; limitation of comparison
Krita	Ordered layer stack with groups, masks, filters, blending modes; dedicated vector layers; multiple brush engines and presets; adjustable sensor curves; visible undo history and cumulative stroke grouping.	A feature-rich 2D painting baseline emphasizes familiar ordered containers and specialized non-destructive tools. Manuals describe user features, not internal file or undo implementation details.
Clip Studio Paint	Vector layer stores editable paths/control points; tools can alter width/shape; vector eraser offers touched-area, intersection, and whole-line modes.	Editable vector strokes benefit from explicit post-stroke tools and erase semantics. Its vector-layer workflow is not a general arbitrary composition graph.
Procreate	Brush Studio exposes detailed attributes, adjustable pressure/tilt behavior, and a live Drawing Pad preview.	Brush UX needs immediate feedback and calibration. This is a brush-configuration comparison, not evidence for node-based document composition.
Photoshop	Public help describes layers and non-destructive techniques including masks, adjustment layers, and Smart Objects.	Non-destructive editing can coexist with a layer-oriented workflow. “Non-destructive” does not require every user action to be stored as a durable event log.

Software / standard	Documented approach relevant here	Useful lesson for Painter; limitation of comparison
Blender	Node groups have defined contexts and outputs; compositor Group Output selects what leaves a group; Grease Pencil supports non-destructive modifier stacks and custom Geometry Nodes modifiers; undo history is user-visible and session-limited.	Useful precedent for typed contextual graphs, explicit output, and modifiers. Blender is a broad 2D/3D content tool, not a direct replacement for a dedicated painting UX.
TouchDesigner	A pull-based operator graph evaluates nodes when requested and when dependencies/parameters provide a reason to cook.	Useful performance model for demand-driven previews and dirty dependencies; not a painting app and not a template for Painter's entire runtime.
SVG 2 / W3C	Document tree and rendering tree can differ; reuse and render order are explicit; compositing is separately specified.	A precise vocabulary for source structure vs rendered result. SVG's interchange model does not capture Painter's proposed brush-history or editing semantics.

13. Source guide

Project-local evidence is in `docs/project-brief.md`, `docs/architecture.md`, `docs/status.md`, ADR 0002, ADR 0004, and ADR 0005. These record current intended scope and architecture; where the brief and accepted ADR/code disagree, the report identifies the conflict rather than treating one as an industry fact.

The external comparisons use official product or standards documentation:

- Krita Manual — Introduction to Layers and Masks. Layer stack, groups, masks, filters, and compositing.
- Krita Manual — Vector Layers. Vector objects, z-order, and object editing.
- Krita Manual — Loading and Saving Brushes. Brush engines, presets, sensor curves, and preview.
- Krita Manual — Undo History. History navigation and cumulative undo behavior.
- Clip Studio Paint User Guide — Vector Layers. Editable vector strokes and vector editing tools.
- Procreate Handbook — Brush Studio. Brush attributes, pressure response, and live preview.
- Adobe Photoshop Help — Non-destructive editing techniques. Publicly documented non-destructive layer workflows.
- Blender Manual — Geometry Node Editor. Node groups and their modifier/tool contexts.
- Blender Manual — Group Output. Explicit output contract for a node group.
- Blender Manual — Grease Pencil Modifiers. Non-destructive modifier stack and applying modifiers.
- Blender Manual — Undo & Redo. Undo timeline, redo truncation, and loss of action history on quit.
- Derivative TouchDesigner Documentation — Cook. Pull-based, demand-driven node evaluation.
- W3C — SVG 2 Rendering Model. Document/render trees, painter's model, and rendering order.
- W3C — Compositing and Blending Level 1. Compositing vocabulary and operations.
- Flutter Documentation — Handling errors in Flutter. Framework and platform-dispatcher error handling.

Research status: comparative synthesis only. No external product is proposed as a direct architectural template; no recommendations in this report are accepted project decisions until recorded through the project's decision process.